

Chapter 11 — Quickselect

0. What You Will Learn in This Chapter

Quickselect is an important selection algorithm used when you need the **k-th smallest**, **k-th largest**, or **top k elements** without fully sorting the array. Sorting solves many ranking problems, but sorting everything costs:

$O(n \log n)$

Quickselect can often find the needed element in average:

$O(n)$

By the end of this chapter, you should understand:

- What Quickselect is
- How it relates to QuickSort
- Why partitioning works
- How to find k-th smallest
- How to find k-th largest
- How to convert k-th largest into an index
- How to use Lomuto partition
- How to use randomized pivoting
- When Quickselect is better than sorting
- When heap is better than Quickselect
- Common off-by-one mistakes
- Quickselect average vs worst-case complexity
- How to explain Quickselect in interviews

Core LeetCode problems covered in this chapter:

- 1.1. Kth Largest Element in an Array
- 2.2. K Closest Points to Origin
- 3.3. Top K Frequent Elements
- 4.4. Find the Kth Largest Integer in the Array
- 5.5. Kth Largest Element in a Stream, heap comparison
- 6.6. Wiggle Sort II, advanced Quickselect use
- 7.7. K Closest Points, Quickselect version
- 8.8. Third Maximum Number, simpler alternative
- 9.9. Relative Ranks, sorting comparison
- 10.10. Sort Colors, partitioning relation

1. What Is Quickselect?

Quickselect is an algorithm for finding the element that would appear at a certain index if the array were sorted.

Example:

nums = {7, 10, 4, 3, 20, 15}

Sorted:

{3, 4, 7, 10, 15, 20}

The 3rd smallest is:

7

The 2nd largest is:

15

Quickselect finds these without fully sorting the array.

2. Why Quickselect Exists

Suppose you only need the k-th largest element.

Sorting the whole array works:

```
sort(nums.begin(), nums.end());  
return nums[nums.size() - k];
```

Time:

$O(n \log n)$

But sorting is doing more work than needed.

If you only need one element, you do not need every element in exact sorted order.

Quickselect uses partitioning to place one pivot into its final sorted position.

Then it decides whether to search left side or right side.

This avoids fully sorting both sides.

3. Quickselect vs QuickSort

Quickselect is related to QuickSort.

QuickSort

QuickSort:

2. Pick pivot.
3. Partition array.
4. Recursively sort left side.
5. Recursively sort right side.

QuickSort must process both sides.

Quickselect

Quickselect:

6. Pick pivot.
7. Partition array.
8. If pivot index is target, return.
9. If target is left of pivot, search left side.
10. If target is right of pivot, search right side.

Quickselect only processes one side.

That is why average time is $O(n)$.

4. Core Partition Idea

Partition rearranges the array so that:

values $\leq$ pivot are on the left
pivot is in its final position
values $>$ pivot are on the right

Example:

nums = {7, 10, 4, 3, 20, 15}
pivot = 7

After partition:

{4, 3, 7, 10, 20, 15}

Pivot 7 is at index 2.
Everything left of it is ≤ 7 .
Everything right of it is > 7 .
So if we need index 2, we are done.

5. When to Use Quickselect

Use Quickselect when the problem asks for:

5.1 K-th Largest or K-th Smallest

Problem clues:

- k-th largest
- k-th smallest
- find rank
- return element with order statistic

Examples:

- Kth Largest Element in an Array
 - Kth Largest Integer in the Array
-

5.2 Top K Elements Where Order Does Not Matter

Problem clues:

- return k closest
- return k most frequent
- return any order

Examples:

- K Closest Points to Origin
- Top K Frequent Elements

Quickselect can partition so the top k elements are on one side.

5.3 Need Better Than Full Sort

If sorting is acceptable, it is simpler.

But if interviewer asks for average $O(n)$, Quickselect is useful.

6. When NOT to Use Quickselect

6.1 Need Fully Sorted Output

Quickselect only guarantees the target element or top k partition.

It does not sort the whole array.

If output must be sorted, use sorting.

6.2 Need Streaming Data

Quickselect works on a fixed array.

For data arriving over time, use heap.

Example:

- Kth Largest Element in a Stream
-

6.3 Need Worst-Case Guarantee

Quickselect average time is $O(n)$, but worst case is $O(n^2)$.

If worst-case guarantee matters, use:

- heap
 - sorting
 - deterministic median-of-medians, rarely used in interviews
-

6.4 K Is Small and Heap Is Simpler

For top k with small k:

$O(n \log k)$

heap may be very practical and easier to reason about.

7. Quickselect vs Sorting vs Heap

Sorting	simple, sorted output	$O(n \log n)$	$O(1) / O(\log n)$
Min/Max Heap	streaming, top k	$O(n \log k)$	$O(k)$
Quickselect	fixed array, average fastest	$O(n)$ average	$O(1)$
Bucket Sort	frequencies with bounded counts	$O(n)$	$O(n)$

8. Mental Models

8.1 Pivot Finds Its Final Seat

After partitioning, pivot is exactly where it would be in a sorted array.

The left and right sides are not fully sorted, but pivot's position is final.

8.2 Search Only the Side That Matters

If pivot index is too small, target must be right.
If pivot index is too large, target must be left.
You ignore the other side.

8.3 K-th Largest as Sorted Index

For zero-indexed ascending sorted order:

kth largest index = $n - k$

Example:

nums sorted = {3, 4, 7, 10, 15, 20}

2nd largest:

$n - k = 6 - 2 = 4$

Index 4 is:

15

9. Pattern Recognition Checklist

Ask:

11. Is the problem asking for k-th largest or k-th smallest?
12. Do I only need one ranked element?
13. Do I need top k elements but not sorted?
14. Is full sorting unnecessary?
15. Is average $O(n)$ acceptable?
16. Is the input fixed, not streaming?
17. Can I compare elements by a score?
18. Can partitioning separate better/worse elements?
19. Does output order not matter?
20. Would sorting be accepted but not optimal?

If yes, Quickselect may apply.

10. C++ Tools Needed

10.1 swap

```
swap(nums[i], nums[j]);
```

Used heavily in partition.

10.2 Random Pivot

```
int pivotIndex = left + rand() % (right - left + 1);
```

Random pivot reduces chance of worst-case behavior.

10.3 nth_element

C++ has built-in Quickselect-like function:

```
nth_element(nums.begin(), nums.begin() + index, nums.end());
```

After this:

- element at index is the element that would be there in sorted order
- elements before it are $\leq$ it
- elements after it are $\geq$ it
- sides are not fully sorted

Example:

```
nth_element(nums.begin(), nums.begin() + n - k, nums.end());  
return nums[n - k];
```

In interviews, it is useful to know, but you should also understand manual Quickselect.

11. Core Templates

Template 1 — Lomuto Partition Ascending

This places pivot in final sorted ascending position.

```
int partition(vector<int>& nums, int left, int right) {  
    int pivot = nums[right];  
    int store = left;  
  
    for (int i = left; i < right; i++) {  
        if (nums[i] <= pivot) {  
            swap(nums[store], nums[i]);  
            store++;  
        }  
    }  
  
    swap(nums[store], nums[right]);  
  
    return store;  
}
```

After partition:

```
nums[left ... store-1] <= pivot  
nums[store] = pivot  
nums[store+1 ... right] > pivot
```

Template 2 — Randomized Partition

```
int partition(vector<int>& nums, int left, int right) {  
    int pivotIndex = left + rand() % (right - left + 1);  
    swap(nums[pivotIndex], nums[right]);
```

```

int pivot = nums[right];
int store = left;

for (int i = left; i < right; i++) {
    if (nums[i] <= pivot) {
        swap(nums[store], nums[i]);
        store++;
    }
}

swap(nums[store], nums[right]);

return store;
}

```

Template 3 — Quickselect by Target Index

```

int quickselect(vector<int>& nums, int targetIndex) {
    int left = 0;
    int right = nums.size() - 1;

    while (left <= right) {
        int pivotIndex = partition(nums, left, right);

        if (pivotIndex == targetIndex) {
            return nums[pivotIndex];
        } else if (pivotIndex < targetIndex) {
            left = pivotIndex + 1;
        } else {
            right = pivotIndex - 1;
        }
    }

    return -1;
}

```

Template 4 — K-th Largest

```

int findKthLargest(vector<int>& nums, int k) {
    int n = nums.size();
    int targetIndex = n - k;
}

```

```

    return quickselect(nums, targetIndex);
}

```

Template 5 — K-th Smallest

```

int findKthSmallest(vector<int>& nums, int k) {
    int targetIndex = k - 1;

    return quickselect(nums, targetIndex);
}

```

Template 6 — Quickselect With Custom Score

For points, compare by distance.

```

int distance(vector<int>& p) {
    return p[0] * p[0] + p[1] * p[1];
}

int partition(vector<vector<int>>& points, int left, int right) {
    int pivotIndex = left + rand() % (right - left + 1);
    swap(points[pivotIndex], points[right]);

    int pivotDist = distance(points[right]);
    int store = left;

    for (int i = left; i < right; i++) {
        if (distance(points[i]) <= pivotDist) {
            swap(points[store], points[i]);
            store++;
        }
    }

    swap(points[store], points[right]);

    return store;
}

```

12. Major Variations of Quickselect

K-th smallest	index k - 1	order statistic

Variation	Goal	Example
K-th largest	index $n - k$	Kth Largest Element
Top K smallest by score	partition first k	K Closest Points
Top K frequent	partition by frequency	Top K Frequent Elements
String numeric ranking	custom compare	Kth Largest Integer
Built-in selection	<code>nth_element</code>	C++ shortcut

13. Full Problem Walkthrough 1 — LeetCode 215: Kth Largest Element in an Array

13.1 Problem Summary

Given an integer array `nums` and integer `k`, return the k -th largest element.

Example:

`nums = {3, 2, 1, 5, 6, 4}`

`k = 2`

Output:

5

Sorted ascending:

`{1, 2, 3, 4, 5, 6}`

2nd largest is 5.

13.2 How to Identify Quickselect

Problem asks:

- k -th largest
- no need to fully sort
- fixed array

Quickselect is appropriate.

13.3 Brute Force / Sorting Solution

```
sort(nums.begin(), nums.end());
return nums[nums.size() - k];
```

Time:

$O(n \log n)$

Works, but Quickselect improves average time.

13.4 Key Conversion

K-th largest in ascending sorted array is at index:

$n - k$

13.5 C++ Code — Manual Quickselect

```
class Solution {
public:
    int partition(vector<int>& nums, int left, int right) {
        int pivotIndex = left + rand() % (right - left + 1);
        swap(nums[pivotIndex], nums[right]);

        int pivot = nums[right];
        int store = left;

        for (int i = left; i < right; i++) {
            if (nums[i] <= pivot) {
                swap(nums[store], nums[i]);
                store++;
            }
        }

        swap(nums[store], nums[right]);

        return store;
    }

    int findKthLargest(vector<int>& nums, int k) {
        int n = nums.size();
        int targetIndex = n - k;

        int left = 0;
        int right = n - 1;

        while (left <= right) {
            int pivotIndex = partition(nums, left, right);

            if (pivotIndex == targetIndex) {
                return nums[pivotIndex];
            } else if (pivotIndex < targetIndex) {
                left = pivotIndex + 1;
            }
        }
    }
};
```

```
        } else {  
            right = pivotIndex - 1;  
        }  
    }  
  
    return -1;  
}  
};
```

13.6 Dry Run Concept

nums = {3,2,1,5,6,4}
k = 2

Target index:

$n - k = 6 - 2 = 4$

We need the element that would be at index 4 if sorted.

Sorted:

{1,2,3,4,5,6}

Index 4 is 5.

Quickselect partitions until pivot lands at index 4.

13.7 Complexity

Average time:

$O(n)$

Worst-case time:

$O(n^2)$

Space:

$O(1)$

for iterative version.

13.8 Why Random Pivot Helps

If pivot is always worst possible, like smallest or largest element, partition removes only one item each time.

That causes $O(n^2)$.

Random pivot makes this unlikely.

13.9 Common Mistakes

Mistake 1: Wrong target index

Wrong for k-th largest:

targetIndex = k - 1;

Correct:

```
targetIndex = n - k;
```

[Mistake 2: Assuming array becomes sorted](#)

Quickselect does not fully sort the array.

[Mistake 3: Forgetting worst case](#)

Average $O(n)$, worst $O(n^2)$.

13.10 STL nth_element Version

```
class Solution {
public:
    int findKthLargest(vector<int>& nums, int k) {
        int n = nums.size();

        nth_element(nums.begin(), nums.begin() + (n - k), nums.end());

        return nums[n - k];
    }
};
```

This is concise and useful in practical C++.

14. Full Problem Walkthrough 2 — LeetCode 973: K Closest Points to Origin

14.1 Problem Summary

Given points and integer k , return k closest points to origin.

Example:

```
points = {{1,3}, {-2,2}}
```

```
k = 1
```

Output:

```
{{-2,2}}
```

14.2 How to Identify Quickselect

Problem asks:

- k closest
- output order does not matter
- no need to sort all points

Quickselect can partition the k closest points into the first k positions.

14.3 Distance Function

Use squared distance:

$$x*x + y*y$$

No square root needed.

14.4 C++ Code — Quickselect

```
class Solution {
public:
    int dist(vector<int>& p) {
        return p[0] * p[0] + p[1] * p[1];
    }

    int partition(vector<vector<int>>& points, int left, int right) {
        int pivotIndex = left + rand() % (right - left + 1);
        swap(points[pivotIndex], points[right]);

        int pivotDist = dist(points[right]);
        int store = left;

        for (int i = left; i < right; i++) {
            if (dist(points[i]) <= pivotDist) {
                swap(points[store], points[i]);
                store++;
            }
        }

        swap(points[store], points[right]);

        return store;
    }

    vector<vector<int>> kClosest(vector<vector<int>>& points, int k) {
        int left = 0;
        int right = points.size() - 1;
        int target = k - 1;

        while (left <= right) {
            int pivotIndex = partition(points, left, right);

            if (pivotIndex == target) {
                break;
            }
        }
    }
};
```

```

        } else if (pivotIndex < target) {
            left = pivotIndex + 1;
        } else {
            right = pivotIndex - 1;
        }
    }

    return vector<vector<int>>(points.begin(), points.begin() + k);
}
};

```

14.5 Complexity

Average time:

$O(n)$

Worst-case:

$O(n^2)$

Space:

$O(1)$

excluding output.

14.6 Sorting Alternative

```

sort(points.begin(), points.end(), [&](vector<int>& a, vector<int>& b) {
    return dist(a) < dist(b);
});

```

Time:

$O(n \log n)$

Simpler, but slower asymptotically.

14.7 Heap Alternative

Use max heap of size k .

Time:

$O(n \log k)$

Good when k is small.

15. Full Problem Walkthrough 3 — LeetCode 347: Top K Frequent Elements

15.1 Problem Summary

Given integer array `nums`, return `k` most frequent elements.

Example:

```
nums = {1,1,1,2,2,3}
```

```
k = 2
```

Output:

```
{1,2}
```

15.2 How Quickselect Applies

First count frequencies.

Then we need top `k` by frequency.

This can be solved by:

- sorting frequencies
- heap
- bucket sort
- Quickselect

Quickselect partitions elements by frequency.

15.3 C++ Code — Quickselect Version

```
class Solution {
public:
    unordered_map<int, int> freq;

    int partition(vector<int>& unique, int left, int right) {
        int pivotIndex = left + rand() % (right - left + 1);
        swap(unique[pivotIndex], unique[right]);

        int pivotFreq = freq[unique[right]];
        int store = left;

        for (int i = left; i < right; i++) {
            if (freq[unique[i]] <= pivotFreq) {
                swap(unique[store], unique[i]);
                store++;
            }
        }
    }
}
```

```

        swap(unique[store], unique[right]);

        return store;
    }

    vector<int> topKFrequent(vector<int>& nums, int k) {
        for (int x : nums) {
            freq[x]++;
        }

        vector<int> unique;

        for (auto& p : freq) {
            unique.push_back(p.first);
        }

        int n = unique.size();
        int target = n - k;

        int left = 0;
        int right = n - 1;

        while (left <= right) {
            int pivotIndex = partition(unique, left, right);

            if (pivotIndex == target) {
                break;
            } else if (pivotIndex < target) {
                left = pivotIndex + 1;
            } else {
                right = pivotIndex - 1;
            }
        }

        return vector<int>(unique.begin() + target, unique.end());
    }
};

```

15.4 Explanation

We partition by frequency ascending.
 After Quickselect at index $n - k$:

lower frequency elements are before target
top k frequency elements are from target to end

The returned top k order does not matter.

15.5 Complexity

Let:

u = number of unique elements

Counting:

$O(n)$

Quickselect average:

$O(u)$

Total average:

$O(n + u)$

Space:

$O(u)$

15.6 Common Mistakes

- Running Quickselect on original nums instead of unique values
 - Sorting when asked for average $O(n)$
 - Returning wrong side of partition
 - Confusing k with target index
-

16. Full Problem Walkthrough 4 — LeetCode 1985: Find the Kth Largest Integer in the Array

16.1 Problem Summary

Given array of numeric strings, return the k -th largest integer.

Numbers may be very large, so they cannot fit in normal integer types.

Example:

`nums = {"3", "6", "7", "10"}`

`k = 4`

Output:

`"3"`

16.2 How to Compare Numeric Strings

For non-negative integer strings:

- 21. Longer length means larger number.
- 22. If same length, lexicographic comparison works.

Example:

"100" > "99"

because length 3 > length 2.

16.3 Sorting Solution

```
class Solution {
public:
    bool smaller(const string& a, const string& b) {
        if (a.size() != b.size()) {
            return a.size() < b.size();
        }

        return a < b;
    }

    string kthLargestNumber(vector<string>& nums, int k) {
        sort(nums.begin(), nums.end(), [&](const string& a, const string& b) {
            return smaller(a, b);
        });

        return nums[nums.size() - k];
    }
};
```

16.4 Quickselect Idea

You can use the same comparator with Quickselect.

Target index:

$n - k$

Partition strings by numeric comparison.

16.5 Complexity

Sorting:

$O(n \log n * m)$

where m is average string length.

Quickselect average:

$O(n * m)$

17. Full Problem Walkthrough 5 — nth_element for K-th Largest

C++ provides `nth_element`, which is essentially selection.

17.1 Basic Usage

```
nth_element(nums.begin(), nums.begin() + index, nums.end());
```

After this:

```
nums[index]
```

is the element that would appear at index in sorted order.

17.2 K-th Largest

```
class Solution {
public:
    int findKthLargest(vector<int>& nums, int k) {
        int index = nums.size() - k;

        nth_element(nums.begin(), nums.begin() + index, nums.end());

        return nums[index];
    }
};
```

17.3 K-th Smallest

```
int kthSmallest(vector<int>& nums, int k) {
    int index = k - 1;

    nth_element(nums.begin(), nums.begin() + index, nums.end());

    return nums[index];
}
```

17.4 Top K Smallest

```
vector<int> topKSmallest(vector<int>& nums, int k) {
    nth_element(nums.begin(), nums.begin() + k, nums.end());

    return vector<int>(nums.begin(), nums.begin() + k);
}
```

Be careful:

If you need exactly first k elements, the nth iterator should often be:

```
nums.begin() + k
```

This places the element at index k correctly, so indices 0 to k - 1 are $\leq$ it.
For k-th element itself, use:

```
nums.begin() + (k - 1)
```

18. Partitioning Deep Dive

18.1 Lomuto Partition

Lomuto partition is simple:

```
pivot = nums[right]
store = left
```

Scan from left to right.

Whenever value belongs on left side, swap it to store.

At end, put pivot at store.

18.2 Example

```
nums = {9, 3, 2, 4, 8}
pivot = 4
```

Start:

```
store = 0
```

Scan:

0	9	no	skip	{9,3,2,4,8}	0
1	3	yes	swap 9 and 3	{3,9,2,4,8}	1
2	2	yes	swap 9 and 2	{3,2,9,4,8}	2
3	4	pivot location before final swap			
final			put pivot at store	{3,2,4,9,8}	2

Pivot index = 2.

19. Common Edge Cases for Quickselect

19.1 Duplicates

Quickselect works with duplicates.
But partition position may vary.

Example:

nums = {3, 2, 3, 1, 2, 4, 5, 5, 6}
k = 4

Sorted:

{1,2,2,3,3,4,5,5,6}

4th largest is 4.

Target index:

$9 - 4 = 5$

19.2 k = 1

K-th largest is max element.

Quickselect still works.

19.3 k = n

K-th largest is min element.

Target index:

0

19.4 Already Sorted Array

Without random pivot, worst-case risk.

Use randomized pivot.

19.5 All Equal Elements

Partition may be unbalanced, but answer is still correct.

Three-way partition can improve this, but standard Quickselect is usually accepted.

20. Three-Way Partition Optimization

If there are many duplicates, normal partition can be inefficient.

Three-way partition splits into:

less than pivot
equal to pivot
greater than pivot

This is Dutch National Flag style.

Concept:

```
lt = left
i = left
gt = right
```

Then:

- if `nums[i] < pivot`: swap with `lt`, increment both
- if `nums[i] > pivot`: swap with `gt`, decrement `gt`
- if equal: increment `i`

After partition:

```
[left ... lt-1] < pivot
[lt ... gt] == pivot
[gt+1 ... right] > pivot
```

If target index lies inside `[lt, gt]`, answer is pivot.

This is useful but more complex.

21. Common Mistakes and Wrong Code

Mistake 1 — Wrong K Conversion

For k-th largest:

```
targetIndex = n - k
```

For k-th smallest:

```
targetIndex = k - 1
```

Mistake 2 — Assuming First K Are Sorted

After Quickselect, first k elements are not sorted.

They are only the k smallest or k largest group depending on partition logic.

Mistake 3 — No Random Pivot

Using the last element as pivot can degrade to $O(n^2)$ on sorted input.

Random pivot is safer.

Mistake 4 — Returning Pivot Too Early

Only return when:

```
pivotIndex == targetIndex
```

Otherwise search the correct side.

Mistake 5 — Wrong Partition Direction

If partition ascending, k-th largest uses index `n - k`.

If partition descending, k-th largest uses index $k - 1$.
Both can work, but do not mix them.

Mistake 6 — Using Quickselect for Streaming

For dynamic stream, use heap.

22. Interview Explanation Templates

22.1 K-th Largest

I convert k-th largest into the index it would occupy in sorted ascending order, which is $n - k$. Then I use Quickselect. Each partition places a pivot in its final sorted position, and I only recurse into the side containing the target index.

22.2 Why Average $O(n)$

Each partition scans the current range once. With a reasonably balanced random pivot, the searched range shrinks by about half on average, so the total work is $n + n/2 + n/4 + \dots$, which is $O(n)$.

22.3 Top K

I do not need the top k elements sorted. I only need them separated from the rest. Quickselect partitions the array so the k desired elements lie on one side, then I return that side.

22.4 Quickselect vs Sorting

Sorting orders every element, which costs $O(n \log n)$. Quickselect only finds the target rank, so it avoids sorting irrelevant parts and runs in average $O(n)$.

23. Complexity Patterns

K-th largest manual Quickselect	$O(n)$ avg, $O(n^2)$ worst	$O(1)$ iterative
K-th smallest	$O(n)$ avg, $O(n^2)$ worst	$O(1)$
K closest points	$O(n)$ avg	$O(1)$ extra
Top K frequent with Quickselect	$O(n + u)$ avg	$O(u)$
Sorting alternative	$O(n \log n)$	$O(1) / O(\log n)$
Heap alternative	$O(n \log k)$	$O(k)$
nth_element	$O(n)$ avg	$O(1)$ extra

24. Time-Boxed Practice Guidance

Intro Quickselect

Target:

20–30 minutes

Problems:

- Kth Largest Element in an Array
- Kth Smallest element in an array, custom practice

Medium Quickselect

Target:

30–45 minutes

Problems:

- K Closest Points to Origin
- Top K Frequent Elements

Advanced / Compare Approaches

Target:

45–60 minutes

Problems:

- Wiggle Sort II
- Kth Largest Integer in the Array
- Top K Frequent Words, usually heap/sort preferred

25. Expanded Practice Set

25.1 Must-Do

- 1.1. Kth Largest Element in an Array
- 2.2. K Closest Points to Origin
- 3.3. Top K Frequent Elements
- 4.4. Find the Kth Largest Integer in the Array

25.2 Sorting / Heap Comparisons

- 5.5. Kth Largest Element in a Stream
- 6.6. Sort Characters By Frequency
- 7.7. Top K Frequent Words
- 8.8. Relative Ranks
- 9.9. Third Maximum Number

25.3 Strong Interview Practice

- 10.10. Wiggle Sort II
- 11.11. K Closest Points to Origin, solve three ways
- 12.12. Top K Frequent Elements, solve with heap, bucket, Quickselect

26. Suggested Study Order

Step 1 — Learn Partition

Practice partitioning arrays manually.

Use:

{9, 3, 2, 4, 8}

and pivot 4.

Goal:

Understand where pivot lands.

Step 2 — K-th Largest

Solve:

- 23. Kth Largest Element using sorting
- 24. Kth Largest Element using heap
- 25. Kth Largest Element using Quickselect

Goal:

Understand tradeoffs.

Step 3 — Top K by Score

Solve:

- 26. K Closest Points using sorting
- 27. K Closest Points using heap
- 28. K Closest Points using Quickselect

Goal:

Learn custom partition score.

Step 4 — Top K Frequency

Solve:

- 29. Top K Frequent Elements using bucket sort
- 30. Top K Frequent Elements using heap
- 31. Top K Frequent Elements using Quickselect

Goal:

Understand frequency ranking.

27. Pattern Decision Table

Need k-th largest in fixed array	Quickselect / heap / sort
Need k-th largest in stream	Heap
Need sorted output	Sorting

Problem Clue	Best Tool
Need top k unordered	Quickselect
Need top k and k is small	Heap
Need average $O(n)$	Quickselect
Need worst-case predictable	Heap or sort
Need frequency top k	Bucket / heap / Quickselect
Need one rank only	Quickselect

28. Final Cheat Sheet

K-th Largest Target Index

```
int targetIndex = nums.size() - k;
```

K-th Smallest Target Index

```
int targetIndex = k - 1;
```

Randomized Partition

```
int partition(vector<int>& nums, int left, int right) {
    int pivotIndex = left + rand() % (right - left + 1);
    swap(nums[pivotIndex], nums[right]);

    int pivot = nums[right];
    int store = left;

    for (int i = left; i < right; i++) {
        if (nums[i] <= pivot) {
            swap(nums[store], nums[i]);
            store++;
        }
    }

    swap(nums[store], nums[right]);

    return store;
}
```

Quickselect Loop

```
while (left <= right) {  
    int pivotIndex = partition(nums, left, right);  
  
    if (pivotIndex == targetIndex) {  
        return nums[pivotIndex];  
    } else if (pivotIndex < targetIndex) {  
        left = pivotIndex + 1;  
    } else {  
        right = pivotIndex - 1;  
    }  
}
```

nth_element

```
nth_element(nums.begin(), nums.begin() + targetIndex, nums.end());  
  
return nums[targetIndex];
```

29. What to Master Before Chapter 12

Before moving to Heap / Priority Queue, make sure you can:

- Explain partitioning
- Implement Lomuto partition
- Use random pivot
- Convert k-th largest to index $n - k$
- Convert k-th smallest to index $k - 1$
- Explain why Quickselect is average $O(n)$
- Explain why worst case is $O(n^2)$
- Know when heap is better
- Know when sorting is simpler
- Solve Kth Largest Element in an Array
- Solve K Closest Points with Quickselect
- Understand that Quickselect does not fully sort the array
- Use `nth_element` as a C++ shortcut

30. Core Takeaway

Quickselect is about finding a rank without fully sorting.

The key idea is:

Partition places one pivot in its final sorted position.

Then:

- if pivot index is the target, return it
- if target is smaller, search left
- if target is larger, search right

For k-th largest:

targetIndex = n - k;

For k-th smallest:

targetIndex = k - 1;

Quickselect is powerful because it gives average $O(n)$ selection.

But remember:

- it does not fully sort
 - it has $O(n^2)$ worst case
 - random pivot helps
 - heap is better for streaming
 - sorting is simpler when $O(n \log n)$ is acceptable
-